📚 C++ Core Concepts & ROS Basics

Brief Overview

This note covers **C++ Core Concepts & ROS Basics** and was created from a 16-page PDF. It dives into **preprocessor** directives, data types, control flow, memory models, object-oriented features, templates, ROS middleware, distance mapping, navigation, and tf transform fundamentals.

Key Points

- Overview of preprocessor directives and macros
- Basics of types, scope, pointers, and arrays
- Object-oriented constructs: structs, classes, inheritance, polymorphism
- ROS concepts: nodes, topics, services, navigation, and tf transforms

---

📂 Preprocessor

> The **preprocessor** runs before the compiler, handling directives that begin with #.

- **#include** – copies the contents of another file (e.g., a header) into the current translation unit.
- **#define** – creates macros, textual substitutions performed inline. Example: #define PI 3.14.
  - *Limitation*: macros ignore C++ scope and type rules; prefer constexpr or const for constants.
- **Conditional compilation** – #ifdef, #ifndef, #else, #endif control which code is compiled.
  - Used for **include guards** to prevent multiple inclusion of a header:

```
#ifndef MY_HEADER_H
#define MY_HEADER_H
// Declarations here...
#endif
```

🧮 Basic Types

> Fundamental data types represent arithmetic values and boolean logic.

Type	Size (typical)	Signed?	Typical Use
char	1 byte	signed/unsigned	characters
short	2 bytes	signed/unsigned	small integers
int	4 bytes	signed/unsigned	general integers
long	≥4 bytes	signed/unsigned	large integers
float	4 bytes	–	single-precision real numbers

double	8 bytes	–	double-precision real numbers
bool	1 byte	–	true/false values

- **Unsigned** types use all bits for magnitude, doubling the positive range compared to their signed counterparts.
- sizeof(type) yields the size in bytes; the exact size (except char) is platform-dependent.

```
int a = 10;
float b = 5.5f;
bool active = true;
unsigned char c = 255; // max 8-bit unsigned value
```

🔁 Flow Control

> Statements that direct the execution order of a program.

- **Conditional branching**: if, else if, else – execute blocks based on boolean expressions.
- **Switch** – selects one of many case labels based on an integral value.
- **Loops**:
  - while – pre-condition check.
  - do-while – post-condition check, runs at least once.
  - for – initializer; condition; increment (standard for iterating over sequences).
  - **Range-based for** (C++11) simplifies iteration over containers.

```
int arr[] = {1, 2, 3};
for (int val : arr) {
    if (val == 2) continue; // skip 2
    // process val
}
```

- **Jump statements**: break (exit loop/switch), continue (skip to next iteration), return (exit function).

📦 Scopes and Visibility

> Determines *where* a name can be used and *who* can see it.

- **Local scope** – variables declared inside {}; destroyed when the block ends.
- **Global scope** – variables declared outside all functions; live for the program's lifetime.
- **Namespaces** – group identifiers (e.g., namespace std) to avoid naming collisions.
- **Shadowing** – an inner declaration with the same name hides an outer one, affecting visibility but not the outer variable's lifetime.

```
int x = 10;          // global
void func() {
```

```
    int x = 5;       // local, shadows global
    {
        int x = 1;   // inner, shadows local
    }
}
```

- **Class visibility** – public, protected, private control member access from outside the class or derived classes.

📍 Pointers

> Variables that store **memory addresses**.

- Declaration: int* p;
- Address-of operator: &var → obtains address.
- Dereference operator: *p → accesses the pointed-to value.

```
int val = 42;
int* ptr = &val;
*ptr = 100;          // val becomes 100
ptr++;               // moves to next integer address (dangerous unless iterating an array)
```

- nullptr (C++11) denotes a null pointer, replacing the older NULL.
- **Common pitfalls**: dereferencing nullptr, using a dangling pointer after the pointed object is freed → leads to segmentation faults.

📚 Arrays

> Fixed-size, contiguous sequences of elements of the same type.

- Array name often **decays** to a pointer to its first element.
- Size is not stored; must be passed separately when arrays are used as function arguments.
- Out-of-bounds access (arr[5] for a 5-element array) results in **undefined behavior**.

```
int arr[3] = {10, 20, 30};
int* p = arr;        // equivalent to &arr[0]
int second = p[1];   // 20
```

🔗 References

> An **alias** for an existing variable, declared with &.

- Must be **initialized** at declaration and cannot be reseated.
- Behave like constant pointers that are automatically dereferenced.

```
void swap(int& a, int& b) {
    int temp = a;
    a = b;
    b = temp;
}
```

- Useful for **pass-by-reference** to avoid copying large objects while still allowing modification (unless passed as const).

📦 Stack vs. Heap

Aspect	Stack	Heap
Allocation	Automatic (function call frames)	Manual (new/delete)
Lifetime	Ends when scope exits	Persists until delete
Speed	Fast (pointer bump)	Slower (allocator)
Size limit	Typically limited	Limited by available RAM
Common errors	Stack overflow	Memory leaks, double free, dangling pointers

```
void func() {
    int stackVar = 10;              // on Stack
    int* heapVar = new int(20);     // on Heap
    delete heapVar;                 // must free manually
}
```

🏗️ Structs with Methods

> In C++, struct and class are nearly identical; the only difference is default member visibility (public for struct).

```
struct Vec2 {
    float x, y;
    float norm() { return sqrt(x*x + y*y); } // method inside struct
};
```

Structs can contain constructors, destructors, and other member functions just like classes.

🛠️ Constructors & Destructors

> Special member functions for object lifecycle management.

- **Constructor** – same name as the class, no return type; runs when an object is created.
- **Copy constructor** – initializes a new object as a copy of an existing one; default is shallow copy if not defined.
- **Destructor** – prefixed with ~; runs when an object goes out of scope or is deleted, responsible for cleanup (e.g., releasing heap memory).

```
class Data {
    int* buf;
public:
    Data() { buf = new int[10]; }          // Constructor
    ~Data() { delete[] buf; }              // Destructor
};
```

📄 Declaration vs. Definition

> **Declaration** introduces a name and its type to the compiler (no storage or body).
> **Definition** provides the actual implementation or allocates storage.

- Example declarations: extern int x;, void f();
- Example definitions: int x = 10;, void f() { … }

One Definition Rule (ODR) – an entity may be declared many times but defined **once**.
Typical layout: header files (.h) contain declarations; source files (.cpp) contain definitions.

➕ Operator Overloading

> Allows user-defined types to customize the behavior of built-in operators.

```
struct Complex {
    float r, i;
    Complex operator+(const Complex& other) {
        return { r + other.r, i + other.i };
    }
};
```

Makes custom types intuitive to use (e.g., C = A + B).

🧬 Inheritance & Polymorphism

- **Inheritance** – derived class acquires members of a base class.
- **Late binding (polymorphism)** – virtual functions enable runtime selection of the appropriate overridden method.

```
class Base { virtual void foo() {} };
class Derived : public Base {};
```

```
Base* b = new Derived();
Derived* d = dynamic_cast(b); // safe downcast, nullptr if not actually Derived
```

- **Dynamic cast** – dynamic_cast(ptr) returns nullptr on failure.
- **Multiple inheritance** – a class can inherit from several bases; may cause the *diamond problem*, resolved with **virtual inheritance**.

🤖 ROS Overview

> **Robot Operating System (ROS)** is a middleware that structures robotics software into **nodes** (processes).

- **Communication**:
  - **Topics** – publish/subscribe (asynchronous).
  - **Services** – request/reply (synchronous).
- **Master** – central coordinator for establishing connections.
- Build tools: **Catkin/CMake**; visualization: **RViz**.

🛠️ Metaprogramming Essentials

Feature	Purpose
Templates	Generic programming; compile-time generation of type-specific code.
Function objects (functors)	Objects with operator(); behave like functions.
using	Modern alias for types (replaces typedef).
constexpr	Forces compile-time evaluation of expressions.
STL	Ready-made containers (vector, list, map), algorithms (sort, find), streams, and smart pointers (shared_ptr, unique_ptr).

📐 Eigen Library

> Header-only C++ template library for linear algebra.

- Uses **expression templates** to avoid temporaries.
- Key types:
  - Eigen::MatrixXd – dynamic-size double matrix.
  - Eigen::Vector3f – fixed-size float vector (3 elements).

```
#include
Eigen::Vector2f v(1.0f, 2.0f);
Eigen::Matrix2f m;
m << 1, 0,
```

```
     0, 1;                // identity
Eigen::Vector2f result = m * v;
```

📏 Distance Map

> A grid where each cell stores the **distance to the nearest obstacle** (0 at obstacles, increasing outward).

- Computed from a binary occupancy grid (e.g., via Chamfer distance or BFS).
- Enables constant-time queries of "how close am I to a wall?" (O(1)).

🧭 Navigation using ROS

- **Goal**: move from start to target while avoiding obstacles.
- Core component: **move_base** node.
  - **Global planner** (e.g., A*) → computes a path on the map.
  - **Local planner** (e.g., Dynamic Window Approach) → generates immediate velocity commands (cmd_vel) respecting dynamic obstacles.
- Inputs: odometry, laser scans, static map.

📍 Localization on a Distance Map

- Estimates robot pose (x, y, θ) using a **particle filter** (e.g., AMCL).
- Employs a **Likelihood Field Model**: for each particle, laser endpoints are projected onto the distance map; cells with low distance values increase the particle's weight.
- Provides smoother, more robust likelihood than binary map matching because small pose errors yield proportionally lower probabilities rather than a hard zero.

🛤️ Planning on a Distance Map

- Path-finding algorithms (Dijkstra, A*) treat the grid as a graph.
- **Cost augmentation**: cells near obstacles receive higher traversal cost (derived from distance map values).
- Resulting paths naturally keep a safety margin from walls, effectively acting as a potential field with obstacle "peaks".

🔄 Isometry (Eigen::Isometry2f / Isometry3f)

> Represents a **rigid body transformation** (rotation + translation) that preserves distances and angles.

- **Mathematical form (2D)**:

$$T = \begin{bmatrix} \cos\theta & -\sin\theta & t_x \\ \sin\theta & \cos\theta & t_y \\ 0 & 0 & 1 \end{bmatrix}$$

- **Construction**: start from identity, set translation and rotation.

```
Eigen::Isometry2f t = Eigen::Isometry2f::Identity();
t.translation() << 10.0f, 5.0f;                      // tx, ty
t.linear() = Eigen::Rotation2Df(M_PI).toRotationMatrix(); // 180° rotation
```

- **Transforming points**: point_global = t * point_local;
- **Chaining**: global_laser_pose = robot_pose * laser_offset; (order matters).
- **Inverse**: robot_pose.inverse() computes the reverse transformation efficiently (transpose rotation, negate translation).

Exam tip: Multiplication is not commutative; robot_pose * laser_offset is correct, the reverse is not meaningful in this context.

🛰️ /odom (Odometry) 📡

> Publishes the robot's estimated pose and velocity using internal sensors (wheel encoders, IMU).

- Message type: nav_msgs/Odometry.
- **Dead-reckoning update**:

$x_{\text{new}} = x_{\text{old}} + v \cdot \Delta t$

- Provides smooth, continuous pose estimates but **drifts** over time.

```
struct Odometry {
    Header header;                // timestamp + frame ("odom")
    string child_frame_id;        // usually "base_link"
    PoseWithCovariance pose;      // (x,y,z) + orientation (quaternion)
    TwistWithCovariance twist;    // linear (vx, vy) + angular (wz) velocities
};
```

📡 /scan (Laser Scan)

> 2-D LiDAR data publishing ranges at fixed angular increments.

- Message type: sensor_msgs/LaserScan.
- **Polar → Cartesian conversion** (robot frame):

\theta_i = \text{angle_min} + i \times \text{angle_increment} $x_i = r_i \cos(\theta_i), \quad y_i = r_i \sin(\theta_i)$

```
struct LaserScan {
    float angle_min;
    float angle_max;
    float angle_increment;
    float range_min, range_max;
    std::vector ranges;
};
```

🌐 tf (Transform Library)

> Manages a **tree of coordinate frames** (world, robot base, sensors, etc.) and provides transformations between any two frames.

- **Frames**:
  - *World/Map*: fixed origin.
  - *Base*: robot's centre.
  - *Laser*: sensor mounting point.
- To locate a laser-detected obstacle in the map, compose transforms:

$$P_{\text{map}} = T_{\text{map}\to\text{odom}}, T_{\text{odom}\to\text{base}}, T_{\text{base}\to\text{laser}}, P_{\text{laser}}$$

🛠️ tf::TransformListener

> Helper class that subscribes to /tf, buffers transforms, and performs lookups.

- **Buffer** stores recent transforms (default ~10 s).
- **lookupTransform(target, source, time, storage)** returns the transformation matrix from *source* to *target*.

```
tf::TransformListener listener; // create once

void laserCallback(const sensor_msgs::LaserScan::ConstPtr& msg) {
    tf::StampedTransform transform;
    try {
        listener.waitForTransform("map", "laser_link", ros::Time(0), ros::Duration(1.0));
        listener.lookupTransform("map", "laser_link", ros::Time(0), transform);

        tf::Vector3 point_laser(2.0, 0.0, 0.0);
        tf::Vector3 point_map = transform * point_laser;
        ROS_INFO("Obstacle at map X: %f, Y: %f", point_map.x(), point_map.y());
    } catch (tf::TransformException &ex) {
        ROS_ERROR("%s", ex.what());
    }
}
```

- **ros::Time(0)** fetches the most recent transform, avoiding timing gaps that cause exceptions.